

KernelLens: Deep Architecture Walkthrough and Mathematical Logic of Triton-to-ONNX/TensorRT Fusion

Antigravity AI Agent
Google DeepMind Team

August 16, 2026

Abstract

This report provides a formal, comprehensive architectural specification and mathematical walkthrough of the `KernelLens` compilation and execution pipeline. `KernelLens` dynamically extracts custom PyTorch Triton kernels, evaluates dynamic execution grids via symbolic AST tracing, handles strict 16-byte CUDA vector memory alignment, maps PTX entry parameters to C++ runtime bindings, and manages transparent dual-way layout transformations (NCHW $\leftrightarrow$ NHWC).

Contents

1	Introduction & System Overview	2
2	System Architecture Pipeline	2
2.1	1. Symbolic Grid Tracing and Manifest Extraction	2
3	Mathematical Logic and Memory Alignment Mechanics	2
3.1	1. Dual-Way Layout Conversions (NCHW $\leftrightarrow$ NHWC)	2
3.2	2. Enforcing 16-Byte Memory Alignment	3
3.3	3. PTX Parameter Binding and Dynamic Shared Memory	3
4	Conclusion	4

1 Introduction & System Overview

The primary objective of **KernelLens** is to bridge the gap between high-level PyTorch models containing custom Triton kernels and production inference engines (ONNX Runtime and NVIDIA TensorRT). High-performance custom Triton kernels often leverage non-standard memory access patterns, dynamic work-grid launches, dynamic shared memory allocations, and implicit stride/layout requirements.

KernelLens automates the compilation and runtime binding through five core stages:

1. **AST & Symbolic Tracing:** Capturing tensor shapes, strides, and dynamic grid dimensions from PyTorch module invocations.
2. **Kernel Manifest Synthesis:** Cataloging input/output tensors, scalar constants, block sizes, and shared memory requirements.
3. **Dynamic Code Generation (ort_gen.py):** Synthesizing C++ CUDA custom operators compliant with ONNX Runtime and TensorRT plugin APIs.
4. **Strict Memory Alignment & Staging:** Enforcing 16-byte alignment ($\text{ptr} \pmod{16} == 0$) and layout transformation kernels.
5. **PTX Launch Integration:** Dynamic parameter packing matching PTX entry signatures and dynamic shared memory allocation via CUDA Driver API calls.

2 System Architecture Pipeline

2.1 1. Symbolic Grid Tracing and Manifest Extraction

When a user module is evaluated, Triton kernel launch grids depend on dynamic input dimensions (e.g., batch size N , spatial dimensions H, W , channel counts C). **KernelLens** intercepts tensor properties by overriding `torch.Tensor.shape` and `torch.Tensor.size()` with symbolic variables $s_0, s_1, s_2, \dots$ powered by SymPy.

Given a dynamic grid tuple $\text{grid}(x, y, z) = (g_x(s_i), g_y(s_i), g_z(s_i))$, the compiler translates symbolic expressions into C++ evaluation code:

$$\text{grid_x} = \max(1u, \lfloor g_x(\text{dim_values}) \rfloor) \quad (1)$$

3 Mathematical Logic and Memory Alignment Mechanics

3.1 1. Dual-Way Layout Conversions (NCHW $\leftrightarrow$ NHWC)

Standard ONNX models exchange tensors in NCHW format (N, C, H, W). However, optimized Triton kernels (such as fused sequential convolutions) compute over NHWC memory ordering (channels_last).

For a tensor with dimensions (N, C, H, W), the linear byte offset under NCHW layout is defined as:

$$\text{Offset}_{\text{NCHW}}(n, c, h, w) = n \cdot (C \cdot H \cdot W) + c \cdot (H \cdot W) + h \cdot W + w \quad (2)$$

Conversely, the physical address offset under NHWC layout is defined as:

$$\text{Offset}_{\text{NHWC}}(n, c, h, w) = n \cdot (H \cdot W \cdot C) + h \cdot (W \cdot C) + w \cdot C + c \quad (3)$$

To maintain numerical compatibility without requiring modifications to the upstream model, `KernelLens` synthesizes high-throughput lightweight CUDA conversion kernels inside the C++ plugin layer:

```

1 __global__ void nchw_to_nhwc_cuda_kernel(const float* __restrict__ src, float*
  __restrict__ dst, int N, int C, int H, int W) {
2     int idx = blockIdx.x * blockDim.x + threadIdx.x;
3     int total = N * C * H * W;
4     if (idx >= total) return;
5     int w = idx % W;
6     int tmp = idx / W;
7     int h = tmp % H;
8     int tmp2 = tmp / H;
9     int c = tmp2 % C;
10    int n = tmp2 / C;
11    int nhwc_idx = n * (H * W * C) + h * (W * C) + w * C + c;
12    dst[nhwc_idx] = src[idx];
13 }

```

Output tensors computed by the kernel in NHWC format are restored back to standard NCHW format using the inverse kernel `nhwc_to_nchw_cuda_kernel`.

3.2 2. Enforcing 16-Byte Memory Alignment

Triton GPU instruction generation utilizes vectorized memory loads (ld.global.v4.b32 or 128-bit SIMD registers). Vectorized 128-bit operations require that starting memory addresses be strictly aligned to 16 bytes:

$$\text{Address} \pmod{16} \equiv 0 \quad (4)$$

If an incoming pointer $\mathbf{P}_{\text{raw}}$ provided by ONNX Runtime or TensorRT fails this condition:

```

1 if (reinterpret_cast<uintptr_t>(raw_ptr) % 16 != 0) {
2     if (aligned_buf == nullptr || aligned_buf_size < nbytes) {
3         if (aligned_buf) cudaFree(aligned_buf);
4         cudaMalloc(&aligned_buf, nbytes);
5         aligned_buf_size = nbytes;
6     }
7     cudaMemcpyAsync(aligned_buf, raw_ptr, nbytes, cudaMemcpyDeviceToDevice,
8     stream);
9     arg_ptr = aligned_buf;
10 }

```

3.3 3. PTX Parameter Binding and Dynamic Shared Memory

Triton’s LLVM NVPTX compiler backend optimizes PTX function signatures by eliminating dead/constant scalar parameters and promoting 64-bit integer address arithmetic.

The generated PTX function header declares explicit argument types:

```

1 .visible .entry _fused_seq_conv_nhwc_kernel(
2     .param .u64 .ptr .global .align 1 _fused_seq_conv_nhwc_kernel_param_0,
3     .param .u64 .ptr .global .align 1 _fused_seq_conv_nhwc_kernel_param_1,
4     .param .u64 .ptr .global .align 1 _fused_seq_conv_nhwc_kernel_param_2,
5     .param .u32 _fused_seq_conv_nhwc_kernel_param_3,
6     ...
7     .param .u64 .ptr .global .align 1 _fused_seq_conv_nhwc_kernel_param_15
8 )

```

To prevent parameter offset corruption during CUDA driver launch, `KernelLens`:

1. Parses the exact PTX entry declaration to determine type bit-widths (.u64 vs .u32).

2. Packs parameter addresses into `std::vector<void*> kp` matching the expected memory width.

3. Configures dynamic shared memory attributes via `cuFuncSetAttribute`:

$$\text{cuFuncSetAttribute}(mKernel, \text{CU_FUNC_ATTRIBUTE_MAX_DYNAMIC_SHARED_SIZE_BYTES}, S_{\text{sh}}) \quad (5)$$

4. Invokes `cuLaunchKernel` with stream synchronization.

—

4 Disambiguating Output Tensor Shapes vs. Launch Grid SymPy

A common point of ambiguity is why `KernelLens` cannot simply use the output tensor’s SymPy shape expression to compute the GPU launch grid dimensions (`grid_z`).

4.1 1. Structural Differences

Property	Output Tensor SymPy Shape	Launch Grid SymPy (<code>grid_fn</code>)
Purpose	Buffer allocation size (<code>cudaMalloc</code>)	GPU hardware block launch count
Scope	High-level model tensor geometry	Internal kernel thread/register tiling
Formula	$(N, 4C, H, W) \implies (1, 512, 64, 64)$	$\text{cdv}(C, \text{BLOCK_SIZE_OC}) \implies 1$

Table 1: Comparison of Output Tensor SymPy vs. Launch Grid SymPy.

4.2 2. Register Unrolling Analysis

Consider a fused sequential convolution kernel where a single thread block unrolls 4 output channel tiles in GPU registers:

$$\text{acc}_0 \leftarrow \text{Channels } [0, 128) \quad (6)$$

$$\text{acc}_1 \leftarrow \text{Channels } [128, 256) \quad (7)$$

$$\text{acc}_2 \leftarrow \text{Channels } [256, 384) \quad (8)$$

$$\text{acc}_3 \leftarrow \text{Channels } [384, 512) \quad (9)$$

Because one block (`pid_oc = 0`) processes all 512 channels internally:

$$\text{grid_z}_{\text{correct}} = \left\lceil \frac{\text{channels}}{\text{BLOCK_SIZE_OC}} \right\rceil = \left\lceil \frac{128}{128} \right\rceil = 1 \quad (10)$$

If the code generator incorrectly derived `grid_z` directly from the output tensor shape (512 channels):

$$\text{grid_z}_{\text{flawed}} = \left\lceil \frac{\text{output_channels}}{\text{BLOCK_SIZE_OC}} \right\rceil = \left\lceil \frac{512}{128} \right\rceil = 4 \quad (11)$$

Launching `grid_z = 4` forces thread block `pid_oc = 3` to attempt reading group 3 (`acc_3`):

$$\text{Channel Index} = 3 \cdot \text{channels} + 3 \cdot \text{channels} = 384 + 384 = 768 \quad (12)$$

Because the weight tensor only has 512 channels, reading index 768 triggers an out-of-bounds illegal address access fault on the GPU, manifesting as `CUDA failure 716: misaligned address`.

4.3 3. Dual-Track SymPy Resolution

To guarantee mathematical correctness, **KernelLens** uses a dual-track SymPy evaluation pipeline:

1. **Tensor Allocation Track:** Uses the output tensor’s SymPy shape expression to evaluate `out_dims` for `ctx.GetOutput(...)`.
2. **Kernel Dispatch Track:** Uses the dynamic lambda function captured directly from the `@triton.jit` invocation tuple to evaluate `(grid_x, grid_y, grid_z)`.

—

5 Conclusion

The **KernelLens** framework achieves end-to-end automation for integrating dynamic PyTorch Triton kernels into enterprise ONNX Runtime and TensorRT pipelines. By combining symbolic dynamic grid resolution, 16-byte CUDA vector alignment, automated layout conversion kernels, and rigorous PTX parameter binding, it provides robust execution stability and maximum inference performance.